Numerical Implementation of MPS and DMRG

Kimi 2.5 and Gemini 3.1

Based on the review article: U. Schollwöck, The density-matrix renormalization group in the age of matrix product states, Annals of Physics 326 (2011) 96-192.

(Dated: April 4, 2026)

This document serves as a concrete numerical implementation guide for translating Matrix Product State (MPS) and Density Matrix Renormalization Group (DMRG) theory into robust, scalable code. It covers practical data structures, canonicalization, truncation, environment construction, explicit MPO formulations, and convergence diagnostics.

I. DATA STRUCTURES

An MPS with open boundary conditions is stored as a list of 3-index tensors:

- $A^{(i)}_{a_{i-1},\sigma_i,a_i}$: Left-canonical tensor at site i, dimensions $(D_{i-1} \times d \times D_i)$
- $B^{(i)}_{a_{i-1},\sigma_i,a_i}$: Right-canonical tensor, dimensions $(D_{i-1} \times d \times D_i)$
- $M^{(i)}_{a_{i-1},\sigma_i,a_i}$: General (non-canonical) tensor during optimization
- Physical dimension d (e.g., $d = 2$ for spin-1/2, $d = 3$ for spin-1)

Storage Convention: For site i, tensor $T^{(i)}$ has shape (D_{i-1}, d, D_i) where the middle index is physical.

Matrix Reshape Convention (Critical): When flattening $T^{(i)}_{a_{i-1},\sigma_i,a_i}$ to matrix $\tilde{M}_{r,c}$ for QR/SVD, the row index r must follow C-order (row-major) mapping:

$$r = a_{i-1} \cdot d + \sigma_i \quad \text{where } a_{i-1} \in [0, D_{i-1} - 1], \sigma_i \in [0, d - 1] \tag{1}$$

Column index $c = a_i$ (right bond). This yields matrix shape $(D_{i-1}d) \times D_i$.

Python/NumPy Implementation:

```
# C-order (row-major) reshape - CRITICAL for index consistency
M_tilde = M.reshape((D_left * d, D_right), order='C')
# Verify: M_tilde[a_prev * d + sigma, a_curr] == M[a_prev, sigma, a_curr]
```

Fortran/MATLAB: Use `reshape(..., order='F')` with reversed index ordering $r = \sigma_i \cdot D_{i-1} + a_{i-1}$ (column-major), or transpose after C-order reshape.

II. CANONICALIZATION ALGORITHMS

A. Left-Canonicalization (QR-based)

More stable than SVD for canonicalization when singular values aren't needed.

Algorithm 1 LeftCanonicalize (Numerically Stable)

Require: MPS tensors $\{M^{(1)}, \ldots, M^{(L)}\}$ with shapes (D_{i-1}, d, D_i)
Ensure: Left-canonical $\{A^{(1)}, \ldots, A^{(L)}\}$, last tensor C (scalar or vector)

$C \leftarrow 1.0$ ▷ Initial transfer matrix, 1×1
for $i = 1$ to $L - 1$ **do**
▷ Contract incoming transfer matrix C with current site
$\tilde{M}_{(a_{i-1}\sigma_i),a_i} \leftarrow \sum_{a'_{i-1}} C_{a_{i-1},a'_{i-1}} M^{(i)}_{a'_{i-1},\sigma_i,a_i}$
▷ For $i = 1$: C is 1×1, so $\tilde{M}$ is simply $M^{(1)}$ reshaped to $(d) \times D_1$
▷ Matrix shape: $(D_{i-1}d) \times D_i$
▷ QR decomposition with column-pivoting for stability
  if $D_{i-1} \cdot d < D_i$ **then**
    Warning: Bond dimension D_i exceeds physical capacity $D_{i-1}d$.
    State is necessarily rank-deficient. Perform truncation or use random initialization.
  end if
  $D_{\text{new}} \leftarrow \min(D_{i-1} \cdot d, D_i)$
  $Q, R \leftarrow \text{QR}(\tilde{M})$ ▷ Economic QR. Q: $(D_{i-1}d) \times D_{\text{new}}$, R: $D_{\text{new}} \times D_i$
  $A^{(i)}_{a_{i-1},\sigma_i,a_i} \leftarrow \text{reshape}(Q, (D_{i-1}, d, D_{\text{new}}))$
  $C \leftarrow R$ ▷ Transfer matrix to next site, shape $D_{\text{new}} \times D_i$
end for
▷ Final site: absorb last C
$A^{(L)}_{a_{L-1},\sigma_L,1} \leftarrow \sum_{a'_{L-1}} C_{a_{L-1},a'_{L-1}} M^{(L)}_{a'_{L-1},\sigma_L,1}$ ▷ Shape: $(D_{L-1}, d, 1)$

B. Mixed-Canonical Form at Bond k

Required for variational minimization.

Algorithm 2 BringToBond

Numerical Note: RQ decomposition (used for right-canonicalization) is less stable than QR and may not be available in all libraries (e.g., PyTorch). Alternative: Reverse the site order, apply LeftCanonicalize, then reverse back.

Require: MPS $\{M^{(i)}\}$, target bond k
Ensure: Left-canonical $A^{(1..k)}$, right-canonical $B^{(k+1..L)}$, center matrix S

Left-canonicalize sites $1 \ldots k$ using QR sweeps
Right-canonicalize sites $L \ldots k+1$ using RQ sweeps (reverse direction)
$S \leftarrow$ matrix at bond k (contains singular values if from SVD)

III. TRUNCATION AND COMPRESSION

A. SVD Compression with Tolerance

Algorithm 3 TruncateBond

Require: Matrix M (dimensions $m \times n$), max bond $D_{\max}$, tolerance ϵ, **abs tolerance** $\epsilon_{\mathbf{abs}} \sim 10^{-14}$
Ensure: Truncated matrices $U, S, V^\dagger$ with $D \leq D_{\max}$

$U, S, V^\dagger \leftarrow \mathrm{SVD}(M, \text{full_matrices=False})$
$s_i \leftarrow \mathrm{diag}(S)$ in descending order
▷ Determine effective D using relative and absolute tolerances
▷ Safety check: prevent $D = 0$ when all singular values vanish
if $s_0 < \epsilon_{\text{abs}}$ **then**
  Warning: All singular values below absolute tolerance.
  Set $D \leftarrow 1$ (keep single largest, even if numerically zero).
else
  $D \leftarrow \min\left(D_{\max}, \mathrm{count}\left(s_i > \max\left(\epsilon \cdot s_0, \epsilon_{\text{abs}}\right)\right)\right)$
  where $\epsilon_{\text{abs}} \sim 10^{-14}$ (double precision) prevents $D \to \infty$ when $s_0 \approx 0$
end if
▷ Truncation error
▷ Discarded weight
$\eta \leftarrow \sum_{i=D}^{r-1} s_i^2$
$U \leftarrow U_{:,0:D-1}$, $S \leftarrow S_{0:D-1,0:D-1}$, $V^\dagger \leftarrow V^\dagger_{0:D-1,:}$
return $U, S, V^\dagger, \eta$

Numerical Note: For $\epsilon \sim 10^{-12}$, single-precision floats may underflow. Use double precision (complex128).

IV. GROUND STATE ALGORITHM: SINGLE-SITE DMRG

A. Effective Environment Construction

The local Hamiltonian at site i is built from pre-computed environments. For numerical implementation, we establish a strict universal index convention to prevent shape mismatches and broadcasting errors:

- **MPO bonds**: `b` (left), `B` (right)
- **MPS bra bonds**: `x` (left), `X` (right)
- **MPS ket bonds**: `y` (left), `Y` (right)
- **Physical indices**: `s` (bra/out), `t` (ket/in)

The environments are always stored as shape `(MPO_bond, Bra_bond, Ket_bond)`. The MPO is stored as `(Left, Right, Out, In)`. In Python, the left environment update $L^{(i)}$ is computed as:

```
# Left Environment Update (site i-1 -> i)
# L_old shape: (b, x, y)
# A_ket shape: (y, t, Y) -> The actual MPS tensor at site i-1
# W shape:     (b, B, s, t) -> The MPO tensor at site i-1
# A_bra shape: (x, s, X) -> The complex conjugate of A_ket
# Output L_new shape: (B, X, Y)

L_new = np.einsum('bxy, ytY, bBst, xsX -> BXY',
L_old, A_ket, W, A_ket.conj(), optimize=True)
```

Note on Performance: Always use `optimize=True` in NumPy's einsum, which automatically finds the optimal contraction path, mimicking the $\mathcal{O}(D^3)$ scaling of step-by-step tensor network contractions.

Similarly, the right environment update $R^{(i)}$ (constructed backwards from L down to 1) is:

```
# Right Environment Update (site i+1 -> i)
# R_old shape: (B, X, Y)
# B_ket shape: (y, t, Y)
# W shape:     (b, B, s, t)
# B_bra shape: (x, s, X)
# Output R_new shape: (b, x, y)

R_new = np.einsum('BXY, ytY, bBst, xsX -> bxy',
R_old, B_ket, W, B_ket.conj(), optimize=True)
```

Verification: After each update, check environment Hermiticity: $\|L - L^\dagger\|_{\max} < 10^{-14}$.
Index Dependency Verification:

- $L^{(i)}$ depends on sites $1 \dots i-1$, hence uses $W^{(i-1)}$ (not $W^{(i)}$).
- $R^{(i)}$ depends on sites $i+1 \dots L$, hence uses $W^{(i+1)}$ (not $W^{(i)}$).
- **Common bug**: Off-by-one error in MPO index. Verify: $L^{(2)}$ uses $W^{(1)}$, $R^{(L-1)}$ uses $W^{(L)}$.

B. Local Optimization (Matrix-Free Formulation)

Solve the eigenvalue problem for the center tensor $M^{(i)}_{\alpha_{i-1},\alpha_i}$ reshaped as vector v of length $dD_{i-1}D_i$:

$$\mathcal{H}_{\text{eff}}\, v = Ev \tag{2}$$

Scalability Note: Explicitly constructing the matrix $\mathcal{H}_{\text{eff}}$ requires $\mathcal{O}(d^2D^4)$ memory and destroys scalability. Therefore, the action of $\mathcal{H}_{\text{eff}}$ on the state vector v must be computed *matrix-free* via sequential tensor contractions. In Python, this is accomplished by wrapping Algorithm 4 in a `scipy.sparse.linalg.LinearOperator` and passing it to `eigsh`. This bypasses the D^4 bottleneck, reducing memory to $\mathcal{O}(D^2)$ and computation time to $\mathcal{O}(D^3)$.

Lanczos stopping criterion: Stop when eigenvalue residual $\|\mathcal{H}_{\text{eff}}v - Ev\| < \delta$ where $\delta \sim 10^{-10}$ or when $N_{\text{iter}} > 20-50$.

Algorithm 4 ApplyEffectiveHamiltonian (Matrix-Free)

Require: Vector v (flattened from $V_{y,t,Y}$), L shape (b,x,y), R shape (B,X,Y), W shape (b,B,s,t)
Ensure: $(\mathcal{H}_{\text{eff}}v)_{x,s,X}$ as flattened vector (maps ket to bra space)
Reshape $v \to V_{y,t,Y}$ ▷ Shape (D_{i-1}, d, D_i)
Step 1 (Contract Left): $T_1 = \sum_y L_{b,x,y}V_{y,t,Y}$
Python: `T1 = np.einsum('bxy, ytY -> bxtY', L, V, optimize=True)`
Step 2 (Contract MPO): $T_2 = \sum_{b,t} T_{1(b,x,t,Y)}W_{b,B,s,t}$
Python: `T2 = np.einsum('bxtY, bBst -> BxsY', T1, W, optimize=True)`
Step 3 (Contract Right): $V_{\text{new}} = \sum_{B,Y} T_{2(B,x,s,Y)}R_{B,X,Y}$
Python: `V_new = np.einsum('BxsY, BXY -> xsX', T2, R, optimize=True)`
return `V_new.flatten()` ▷ Pass back to Lanczos/Davidson solver

Cost: $\mathcal{O}(D_W D_{i-1} D_i d^2 + D_W^2 D_{i-1} D_i d)$.

Numerical Note on Hermiticity: Due to floating-point accumulation during sweeps, the effective Hamiltonian may lose exact machine-precision Hermiticity. This often causes iterative solvers like `scipy.sparse.linalg.eigsh` to fail. However, you must **not** explicitly symmetrize the individual environments L and R (e.g., $L \leftarrow \frac{1}{2}(L + L^\dagger)$). The individual MPO bond channels (like $\hat{S}^+$) are strictly non-Hermitian; symmetrizing them corrupts the Hamiltonian. Instead, simply pass `ishermitian=True` (or explicitly provide `rmatvec=matvec`) when defining the `LinearOperator` so SciPy mathematically enforces Hermiticity internally during Lanczos iterations.

C. Environment Initialization

Before sweeping, initialize boundary environments consistent with the MPO structure:

$$L^{(1)}_{b_0}(a_0, a'_0) = \delta_{b_0, b_{\text{left}}} \delta_{a_0, a'_0} \qquad \text{(Left boundary: } 1 \times 1 \text{ matrices)} \tag{3}$$

$$R^{(L)}_{b_L}(a_L, a'_L) = \delta_{b_L, b_{\text{right}}} \delta_{a_L, a'_L} \qquad \text{(Right boundary: } 1 \times 1 \text{ matrices)} \tag{4}$$

where b_{left} and b_{right} are the MPO indices selecting the Hamiltonian scalar. In code, this means $L^{(1)}$ is a $(D_W, 1, 1)$ tensor with a 1.0 at index b_{left} and 0 elsewhere.

D. Complex Arithmetic Handling

Even for real Hamiltonians (e.g., Heisenberg), maintain `complex128` dtype throughout:

- **Left environment:** Uses complex conjugate only (no transpose): Python: `A.conj()`, **not** `A.T.conj()`.
- **Right environment:** Uses complex conjugate only when performing the explicit `np.einsum` specified above.
- **Verification:** The density matrix $\rho = MM^\dagger$ must be Hermitian. Check $\|\rho - \rho^\dagger\|_{\max} < 10^{-14}$.

E. Datatype and Library Requirements

Floating Point Precision: All tensors must use `complex128` (double precision). Single precision (`float32`) leads to underflow in $\epsilon \sim 10^{-12}$ truncations.

QR Decomposition: Standard `numpy.linalg.qr` lacks column-pivoting. Use:

```
from scipy.linalg import qr
Q, R, P = qr(M, pivoting=True, mode='economic')
# Reorder Q according to permutation P if necessary
```

Ensure diagonal elements of R have non-negative real parts (multiply column j of Q by $\text{sign}(R_{jj})$ if needed).

SVD Truncation: Explicitly sort singular values descending:

```
U, s, Vh = np.linalg.svd(M, full_matrices=False)
# Ensure descending (NumPy default) - reverse if using different library
idx = np.argsort(s)[::-1]
s, U, Vh = s[idx], U[:, idx], Vh[idx, :]
```

F. Sweep Algorithm

Algorithm 5 SingleSiteDMRG (with Gauge Control)

Require: Hamiltonian MPO $\{W^{(i)}\}$, initial MPS guess $\{M^{(i)}\}$, max bond $D_{\max}$, tolerance ϵ
Ensure: Ground state energy E_0 and optimized MPS in mixed-canonical form

```
Right-canonicalize initial MPS using RQ sweeps (or reverse QR)
Initialize all R^(i) environments via backward contraction from site L
L^(1) ← left boundary tensor (see initialization above)
while not converged do
    Right sweep (i = 1 to L − 1):
    for i = 1 to L − 1 do
        Build H_eff from L^(i), R^(i), W^(i)
        Solve H_eff v = E v using ApplyEffectiveHamiltonian and Lanczos
        Reshape v → M^(i)_{a_{i−1},σ_i,a_i}
                                    ▷ Gauge update: SVD and shift center to the right
        Reshape M^(i) → M̃_{(a_{i−1}σ_i),a_i} (matrix (dD_{i−1}) × D_i)
        U, S, V† ← SVD(M̃), truncate to D_max
        D_new ← min(D_max, len(S))
        A^(i) ← reshape(U, (D_{i−1}, d, D_new))                    ▷ Left-canonical
                                    ▷ Absorb singular values into next site (center moves right)
        M^(i+1)_{a_i,σ_{i+1},a_{i+2}} ← Σ_{a'_i,a''_i} diag(S)_{a_i,a'_i} V†_{a'_i,a''_i} M^(i+1)_{a''_i,σ_{i+1},a_{i+2}}
        Update L^(i+1) using A^(i) and W^(i)
    end for
    Left sweep (i = L to 2):
    for i = L to 2 do
        Solve eigenproblem using updated environments
        Reshape M^(i) → M̃_{a_{i−1},(σ_i a_i)} (matrix D_{i−1} × (dD_i))
        U, S, V† ← SVD(M̃), truncate to D_max
        D_new ← min(D_max, len(S))
        B^(i) ← reshape(V†, (D_new, d, D_i))                       ▷ Right-canonical
                                    ▷ Absorb singular values into previous site (center moves left)
        M^(i−1)_{a_{i−2},σ_{i−1},a_{i−1}} ← Σ_{a'_{i−1},a''_{i−1}} M^(i−1)_{a_{i−2},σ_{i−1},a''_{i−1}} U_{a''_{i−1},a'_{i−1}} diag(S)_{a'_{i−1},a_{i−1}}
        Update R^(i−1) using B^(i) and W^(i)
    end for
    Check convergence (ALL must hold):
      1. Energy: |ΔE| = |E_n − E_{n−1}| < ε_E (e.g., 10^−8 per site)
      2. Variance: σ^2 = ⟨ψ|H^2|ψ⟩ − E^2 < ε_σ (stricter, ∼ 10^−6)
      3. State convergence: max_i ‖A^(i)_new − A^(i)_old‖_Frob < ε_tensor
end while
```

Critical Implementation Note — The Local Minimum Problem: Single-site updates with fixed bond dimension D cannot escape local minima during phase transitions or for systems with symmetries. Two mitigation strategies:

1. **Two-Site DMRG**: Use Algorithm 6 for the first 3-5 sweeps to dynamically increase D, then switch to single-site for speed.

2. **Perturbation/Noise Injection**: Before truncation, add random noise to the singular values or density matrix.

V. TWO-SITE DMRG (ROBUST AGAINST LOCAL MINIMA)

For systems with symmetries or near phase transitions, single-site DMRG may get stuck. Two-site updates allow bond dimension adaptation.

Critical Initialization Note: When initializing the random MPS for Two-Site DMRG, do not artificially restrict the initial state to a small bond dimension (e.g., $D = 2$) with the expectation that it will organically grow. For Hamiltonians with strict symmetries (like $SU(2)$ Heisenberg), a small random state may become trapped in a specific symmetry sector. In such cases, the local Effective Hamiltonian has zero matrix elements mapping to other sectors,

causing the SVD to yield exact zeroes for new singular values, completely preventing D from growing. Always initialize the random starting tensors to the full target capacity $D_{\max}$ to ensure the algorithm can explore the full Hilbert space.

Algorithm 6 TwoSiteUpdate

Effective Hamiltonian on two-site space:
$\mathcal{H}_{\text{eff}}^{(i,i+1)} = \sum_{b_{i-1},b_{i+1}} L_{b_{i-1}} \otimes W^{(i)}_{b_{i-1},b_i} \otimes W^{(i+1)}_{b_i,b_{i+1}} \otimes R_{b_{i+1}}$
Dimensions: $d^2 D_{i-1} D_{i+1} \times d^2 D_{i-1} D_{i+1}$.

Require: Current bond dimensions D_{i-1}, D_i, max bond $D_{\max}$, tolerance ϵ

Contract: $\Theta_{\alpha_{i-1},\sigma_i,\sigma_{i+1},\alpha_{i+1}} = \sum_{\alpha_i} A^{(i)}_{\alpha_{i-1},\sigma_i,\alpha_i} B^{(i+1)}_{\alpha_i,\sigma_{i+1},\alpha_{i+1}}$
Reshape: $\Theta \to \Theta_{(\alpha_{i-1}\sigma_i),(\sigma_{i+1}\alpha_{i+1})}$ (matrix of size $(dD_{i-1}) \times (dD_{i+1})$)
Build $\mathcal{H}_{\text{eff}}$ on combined two-site space using sequential contractions.
Solve for lowest eigenvector $\tilde{\Theta}$ (size $d^2 D_{i-1} D_{i+1}$) using Lanczos/Davidson
Reshape $\tilde{\Theta} \to$ matrix $(dD_{i-1}) \times (dD_{i+1})$
SVD: $\tilde{\Theta} = USV^\dagger$
Truncate to $D \le D_{\max}$ singular values using TruncateBond
$A^{(i)} \leftarrow \text{reshape}(U, (D_{i-1}, d, D))$ ▷ Left-canonical
$B^{(i+1)} \leftarrow \text{reshape}(SV^\dagger, (D, d, D_{i+1}))$ ▷ Right-canonical
return Updated tensors, truncation error $\eta = \sum_{\text{discarded}} s_i^2$

VI. MPO CONSTRUCTION FOR HAMILTONIANS

A. From Operators to 4D Tensors

In the literature, MPOs are often written as matrices of operators $\hat{W}_{b_{i-1},b_i}$. To translate this into a 4D array $W^{(i)}$ of shape (D_W, D_W, d, d) required for coding, one evaluates the matrix elements in the local physical basis $|\sigma\rangle$:

$$W^{(i)}_{b_{i-1},b_i,\sigma,\sigma'} = \langle\sigma|\hat{W}_{b_{i-1},b_i}|\sigma'\rangle \tag{5}$$

For practical implementation, define the local spin matrices explicitly. For spin-1/2 ($d = 2$):

$$S^+ = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad S^- = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}, \quad S^z = \begin{pmatrix} 0.5 & 0 \\ 0 & -0.5 \end{pmatrix}, \quad \mathbb{I} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \tag{6}$$

For spin-1 ($d = 3$):

$$S^x = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \quad S^y = \frac{i}{\sqrt{2}} \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}, \quad S^z = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & -1 \end{pmatrix} \tag{7}$$

B. Nearest-Neighbor Heisenberg Model

MPO bond dimension $D_W = 5$. Define operator-valued matrices $\hat{W}^{(i)}$:

$$\hat{W}^{(i)} = \begin{pmatrix} \hat{\mathbb{I}} & \cdot & \cdot & \cdot & \cdot \\ \hat{S}^+ & \cdot & \cdot & \cdot & \cdot \\ \hat{S}^- & \cdot & \cdot & \cdot & \cdot \\ \hat{S}^z & \cdot & \cdot & \cdot & \cdot \\ -h\hat{S}^z & \frac{J}{2}\hat{S}^- & \frac{J}{2}\hat{S}^+ & J_z\hat{S}^z & \hat{\mathbb{I}} \end{pmatrix} \tag{8}$$

Boundary conditions (0-indexed array convention):

- Site 1 (left boundary): $W^{(1)}$ is a $(1, D_W, d, d)$ tensor selecting the last row (index 4 for $D_W = 5$).
  Code: `W_left[0, :, :, :] = bulk_W[4, :, :, :]`
- Site L (right boundary): $W^{(L)}$ is a $(D_W, 1, d, d)$ tensor selecting the first column (index 0).
  Code: `W_right[:, 0, :, :] = bulk_W[:, 0, :, :]`

Index Verification: MPO bond indices $b_i \in \{0, 1, \ldots, D_W - 1\}$. For Heisenberg with $D_W = 5$:

- $b_{\text{left}} = 4$ (index of $\hat{\mathbb{I}}$ in last row)
- $b_{\text{right}} = 0$ (index of $\hat{\mathbb{I}}$ in first column)

Boundary environments: $L_{b_0}^{(1)} = \delta_{b_0,4}$, $R_{b_L}^{(L)} = \delta_{b_L,0}$.

C. AKLT Model

The AKLT model is a spin-1 ($d = 3$) system involving a biquadratic interaction:

$$H_{\text{AKLT}} = \sum_{i=1}^{L-1} \left[\vec{S}_i \cdot \vec{S}_{i+1} + \frac{1}{3} (\vec{S}_i \cdot \vec{S}_{i+1})^2 \right] \tag{9}$$

To construct a highly scalable exact MPO, we cast the interaction as a sum of outer products. Define the 3-component vector of spin operators $\mathbf{S} = (S^x, S^y, S^z)^T$. Define $\mathbf{Q}$ as the 9-component vector containing all pairwise products $Q^{ab} = S^a S^b$ for $a, b \in \{x, y, z\}$. The nearest-neighbor interaction perfectly decouples into $\mathbf{S}_i \cdot \mathbf{S}_{i+1} + \frac{1}{3}\mathbf{Q}_i \cdot \mathbf{Q}_{i+1}$.

The resulting MPO tensor $\hat{W}^{(i)}$ is elegantly expressed as a block matrix with bond dimension $D_W = 1+3+9+1 = 14$:

$$\hat{W}^{(i)} = \begin{pmatrix} \hat{\mathbb{I}} & \mathbf{0}_{1\times 3} & \mathbf{0}_{1\times 9} & 0 \\ \mathbf{S} & \mathbf{0}_{3\times 3} & \mathbf{0}_{3\times 9} & \mathbf{0}_{3\times 1} \\ \mathbf{Q} & \mathbf{0}_{9\times 3} & \mathbf{0}_{9\times 9} & \mathbf{0}_{9\times 1} \\ 0 & \mathbf{S}^T & \frac{1}{3}\mathbf{Q}^T & \hat{\mathbb{I}} \end{pmatrix} \tag{10}$$

Boundary Initialization: For the left boundary $W^{(1)}$, select the bottom row ($b_{\text{left}} = 13$). For the right boundary $W^{(L)}$, select the first column ($b_{\text{right}} = 0$).
Code: `W_left[0, :, :, :] = bulk_W[13, :, :, :]` and `W_right[:, 0, :, :] = bulk_W[:, 0, :, :]`.

VII. COMPRESSION OF MPO-MPS PRODUCT

When applying MPO to MPS, bond dimension grows to $D \cdot D_W$. Compress variationally:

Algorithm 7 VariationalMPSCompression

Require: Target MPS $|\psi\rangle$ (bond D_ψ), initial guess $|\phi\rangle$ (bond $D_\phi < D_\psi$)
Ensure: $|\phi\rangle \approx |\psi\rangle$ with minimal $\|\phi - \psi\|^2$

```
Bring |φ⟩ to mixed-canonical form at bond k = 1 (right-canonicalized)
Compute initial environments L_ψφ, R_ψφ (overlaps between ψ and φ)
while not converged do
    for i = 1 to L − 1 do                                   ▷ Right sweep
        T_{a_{i−1},σ_i,a_i} ← Σ_{a'} L_ψφ · M_ψ^(i) · R_ψφ
        if |φ⟩ is kept strictly in mixed-canonical form at site i then
            M_φ^(i) ← T_{a_{i−1},σ_i,a_i}                   ▷ Metric N = I, avoids costly matrix inversion!
        else
            N_{a_{i−1},a_i} ← Σ_{b_i} L_{b_i}^(i) R_{b_i}^(i)   ▷ Metric, shape D_φ × D_φ
            M_φ^(i) ← Reshape(N^{−1} · Flatten(T))          ▷ Pseudo-inverse if singular
        end if
        SVD and shift gauge right (as in DMRG)
    end for
    Left sweep (analogous)
end while
```

VIII. MODEL EXAMPLES AND PHYSICS

A. Spin-1/2 Heisenberg Chain

The ground state exhibits quasi-long-range antiferromagnetic order (power-law decay of correlations $\sim (-1)^r/r$ at criticality).

Numerical Notes: The Heisenberg ground state is not exactly representable by a finite-D MPS. Due to its critical nature, the entanglement entropy scales logarithmically with system size, requiring bond dimensions $D \sim L^\alpha$ (with $\alpha \approx 1/6$) to maintain accuracy. Results must be extrapolated in the truncation error η or $1/D$.

B. AKLT Model (Exact MPS Representation)

Unlike the Heisenberg model, the AKLT ground state is gapped and can be represented *exactly* as a translationally invariant MPS with bond dimension $D = 2$.

Explicit MPS Matrices: The left-canonical matrices A^σ (where $\sigma \in \{+, 0, -\}$ corresponds to $m_z = +1, 0, -1$) are given by

$$A^+ = \begin{pmatrix} 0 & \sqrt{2/3} \\ 0 & 0 \end{pmatrix}, \tag{11}$$

$$A^0 = \begin{pmatrix} -1/\sqrt{3} & 0 \\ 0 & 1/\sqrt{3} \end{pmatrix}, \tag{12}$$

$$A^- = \begin{pmatrix} 0 & 0 \\ -\sqrt{2/3} & 0 \end{pmatrix}. \tag{13}$$

These satisfy the left-normalization condition $\sum_\sigma A^{\sigma\dagger} A^\sigma = \mathbb{I}_2$.

Verification Test: After implementing the MPS class, verify:

$$\sum_{\sigma\in\{+,0,-\}} A^{\sigma\dagger} A^\sigma = \mathbb{I}_2, \quad \sum_\sigma A^\sigma A^{\sigma\dagger} = \frac{2}{3}\mathbb{I}_2 + \frac{1}{3}\sigma_z \neq \mathbb{I} \tag{14}$$

The left-canonical condition is satisfied; the right-canonical is not (this is an *injective* MPS). For two-site DMRG initialization with $D = 2$ and the MPO provided in Sec. VI C, the energy should converge to exactly $E_0 = -(L-1)\cdot\frac{2}{3}$ with truncation error $\eta \equiv 0$.

Properties:

- **Correlation Functions:** Spin-spin correlations decay exponentially as $\langle S_i^z S_j^z\rangle \sim (-1/3)^{|i-j|}$ with correlation length $\xi = 1/\ln 3 \approx 0.91$.
- **Hidden Order:** The model exhibits a non-local "string order" $\langle S_i^z \exp(i\pi \sum_{k=i+1}^{j-1} S_k^z) S_j^z\rangle = -4/9$ for $|i-j| > 2$, indicating topological order.
- **Open Boundaries:** For OBC, the exact ground state is $|\psi\rangle = \sum_{\{\sigma\}} A^{\sigma_1}\cdots A^{\sigma_L}|\sigma_1\cdots\sigma_L\rangle$, resulting in two free $S = 1/2$ edge spins.

This model serves as an excellent benchmark for DMRG implementations: with $D = 2$, the algorithm should reproduce the exact energy $E_0 = -(L-1)\cdot 2/3$ (for $J = 1$) and exponentially decaying correlations without truncation error.

IX. CONVERGENCE DIAGNOSTICS

- **Energy convergence**: $\Delta E = |E_n - E_{n-1}| < 10^{-8}$ (per site)
- **Truncation error**: Monitor $\eta = \sum_{\text{discarded}} s_i^2$. For accurate results, extrapolate $E(\eta) \to \eta \to 0$.
- **Variance**: $\sigma^2 = \langle\psi|H^2|\psi\rangle - \langle\psi|H|\psi\rangle^2$. Should be $\sim D^{-2}$ for gapped systems.
- **Entanglement entropy**: Check $S = -\sum s_i^2 \ln s_i^2$ saturation across bonds.

Computational Note on Variance: Computing $\langle H^2 \rangle$ by squaring the MPO (bond dimension D_W^2) is inefficient. Instead:

1. Compute $|\phi\rangle = H|\psi\rangle$ via MPO-MPS multiplication (cost $\sim LD_W D^3$).
2. Compute $\langle H^2 \rangle = \langle \phi | \phi \rangle$ (MPS overlap).
3. Reuse $\langle \psi | H | \psi \rangle$ from DMRG energy for E^2 term.

This avoids the D_W^2 scaling of explicit H^2 construction.

X. PRACTICAL IMPLEMENTATION NOTES (PYTHON/NUMPY)

While the mathematical algorithms provide the rigorous framework for DMRG, translating them into high-performance Python code requires navigating several realities of array programming and memory management. The following details are critical for a robust, production-ready codebase.

A. Eigensolver Initial Guess (`v0`)

In Algorithm 5, solving the effective Hamiltonian is the primary computational bottleneck. When using iterative Lanczos/Davidson solvers like `scipy.sparse.linalg.eigsh`, you **must** pass the tensor from the previous sweep as the initial guess vector. If omitted, SciPy initializes a random vector, increasing the number of iterations (and thus computation time) by an order of magnitude.

```
from scipy.sparse.linalg import LinearOperator, eigsh

# Define the matrix-vector product wrapper
def matvec(v):
# Reshape v, apply contractions (Algorithm 4), return flattened result
pass

Heff = LinearOperator((dim, dim), matvec=matvec)

# CRITICAL: Pass the current flattened tensor as the initial guess 'v0'
evals, evecs = eigsh(Heff, k=1, which='SA', v0=M_current.flatten(), tol=1e-10)
M_optimized = evecs[:, 0].reshape(shape, order='C')
```

B. Gauge Shifting via `tensordot`

In the gauge update step of DMRG (absorbing singular values into the adjacent site), the operation is mathematically $M^{(i+1)} = \sum \mathrm{diag}(S) \cdot V^\dagger \cdot M^{(i+1)}$. Do not use `np.einsum` for this matrix-tensor multiplication. Instead, use `np.tensordot`, which maps directly to highly optimized BLAS `dgemm`/`zgemm` routines.

```
# 1. Form the gauge matrix to push right
gauge_matrix = np.diag(S) @ Vh  # Shape: (D_new, D_right)

# 2. Push into the next site using tensordot
# axes=([1], [0]) sums over the 2nd index of gauge and 1st index of M_next
M_next = np.tensordot(gauge_matrix, M_next, axes=([1], [0]))
```

C. Environment Caching

To maintain the strict $\mathcal{O}(1)$ per-site scaling of the sweep algorithm (excluding the internal D^3 tensor contractions), environments must not be recomputed from scratch. Maintain two caches (e.g., Python lists) for the left and right environments.

```
# Initialize caches. L_env[i] holds the left environment acting ON site i.
L_env = [None] * L
R_env = [None] * L

# Initialize boundaries (0-indexed)
L_env[0] = np.zeros((D_w, 1, 1), dtype=np.complex128)
L_env[0][b_left, 0, 0] = 1.0

R_env[L-1] = np.zeros((D_w, 1, 1), dtype=np.complex128)
R_env[L-1][b_right, 0, 0] = 1.0

# Pre-calculate R_env for the initial right-to-left sweep
for i in range(L-1, 0, -1):
R_env[i-1] = update_right_env(R_env[i], B[i], W[i])
```

When moving right (from site i to $i+1$), calculate the new environment using `L_env[i]` and save the result directly to `L_env[i+1]`.

D. Memory Views vs. Copies in `reshape`

In NumPy, `M.reshape(...)` frequently returns a **view** of the original memory buffer rather than a copy. If you reshape a tensor to a matrix, perform an SVD, and then modify the matrix in-place, you risk silently corrupting the MPS state. To be safe, assign newly formed tensors directly rather than modifying slices:

```
# Safe: Assigns the new array object to the list index
MPS[i] = U.reshape((D_left, d, D_new), order='C')

# Dangerous: Modifies the underlying memory buffer in-place
MPS[i][:] = U.reshape((D_left, d, D_new), order='C')
```

E. Robust SVD via SciPy

Always prefer `scipy.linalg.svd` over `numpy.linalg.svd`. SciPy defaults to the LAPACK divide-and-conquer driver (`gesdd`), which is substantially faster for the mid-sized matrices ($D \sim 1000$) encountered in DMRG. However, `gesdd` can occasionally fail to converge for highly degenerate matrices. A robust implementation catches this and falls back to the standard, highly stable `gesvd` driver.

```
from scipy.linalg import svd, LinAlgError

try:
U, S, Vh = svd(M, full_matrices=False)
except LinAlgError:
# Fallback to standard driver if divide-and-conquer fails
U, S, Vh = svd(M, full_matrices=False, lapack_driver='gesvd')
```

F. The `np.einsum` Overhead in Iterative Solvers

While `np.einsum(..., optimize=True)` is highly readable for tensor contractions, it carries significant Python interpreter overhead because it calculates the optimal contraction path on every single call. Inside the `ApplyEffectiveHamiltonian` wrapper—which the Lanczos solver calls hundreds of times per site—you must replace `einsum` with explicit `np.tensordot` and `transpose` operations. This bypasses the Python interpreter and pushes the $\mathcal{O}(D^3)$ heavy lifting directly to pre-compiled C/Fortran BLAS routines, frequently yielding a $> 100\times$ speedup for the overall DMRG sweep.

G. Eigensolver Fallbacks for Exact Degeneracies

For small subsystem dimensions ($D \leq 64$ at the boundaries), it is standard to bypass Lanczos and use dense exact diagonalization (`scipy.linalg.eigh`). However, for topologically ordered states like the AKLT model, exact ground-state degeneracies (e.g., from fractionalized edge spins) can cause the LAPACK divide-and-conquer driver to crash with `LinAlgError: Eigenvalues did not converge`. Robust codes must catch this exception and fall back to the general non-Hermitian solver (`scipy.linalg.eig`), sorting the real parts of the eigenvalues manually.

XI. IMPLEMENTATION VERIFICATION CHECKLIST

The following phased testing protocol validates correct implementation of each subsystem before proceeding to full DMRG. Implement in order; failures at early stages indicate index ordering or conjugation bugs that must be fixed before testing variational optimization.

TABLE I. Phased Implementation Tests

Phase	**Test**	**Expected Result**
1. `Canonicalization`	QR on random MPS $\rightarrow$ LeftCanonicalize	Check $\sum_\sigma A^{(i)\sigma\dagger} A^{(i)\sigma} = \mathbb{I}_{D_i}$ for $i < L$ (within 10^{-14}). Last site is vector $(D_{L-1}d) \times 1$.
2. `Truncation`	SVD compression of AKLT state ($D_{\max} = 2$)	$\eta = 0$ (exact representation maintained). Bond dimension remains $D = 2$.
3. `Environments`	Build $L^{(i)}, R^{(i)}$ for AKLT with identity MPO	Environments proportional to identity on auxiliary space. Verify $L^{(i)} \propto \delta_{a_{i-1}, a'_{i-1}}$.
4. `SingleSite`	AKLT model, $L = 10$, $D = 2$, 2 sweeps	Ground state energy $E_0 = -(L-1) \cdot 2/3 = -6.0$ (machine precision, $\Delta E < 10^{-12}$) using the explicit MPO from Sec. VI C.
5. `TwoSite`	Heisenberg $L = 20$, $D : 4 \rightarrow 20$, $\epsilon = 10^{-10}$	Energy extrapolates to -8.772 (Bethe ansatz). Bond dimension adapts dynamically.
6. `Convergence`	Run single-site DMRG on gapped system	Variance $\sigma^2 < 10^{-6}$ (strictest test). Check $\sigma^2 \sim D^{-2}$ scaling.

A. Common Failure Modes

If tests fail, check these typical implementation errors:

- **Phase 1 fails (non-orthogonal A)**: Reshape index ordering incorrect. Verify row-major vs column-major convention (Section 1).
- **Phase 2 fails ($\eta > 0$ for AKLT)**: Incorrect SVD truncation logic or singular value sorting (descending order required).
- **Phase 3 fails (wrong environments)**: Missing complex conjugate on $A^{(i)*}$ or $B^{(i)*}$ in equations. Check environment indexing (off-by-one error $W^{(i)}$ vs $W^{(i\pm1)}$).
- **Phase 4 fails (wrong AKLT energy)**: MPO boundary conditions incorrect. Verify b_{left} and b_{right} select scalar component.
- **Phase 5 fails (stuck in local minimum)**: Insufficient bond dimension growth or missing noise injection.
- **Phase 6 fails (variance σ^2 large)**: Lanczos solver tolerance too loose or $\mathcal{H}_{\text{eff}}$ matrix-vector product bug (check einsum indices).

B. Reference Values for Heisenberg Chain

For the $S = 1/2$ Heisenberg model with open boundary conditions, exact diagonalization values for small L:

L	E_0/J
4	$-1.616025403784...$
6	$-2.493577131515...$
8	$-3.374932109083...$
10	$-4.258035207...$

Use these to verify MPO construction before attempting DMRG on larger systems.